

Zebrafish Retina Regeneration

Single-Cell RNA-Seq Analysis • Student Project Guide (Project 1)

Table of contents

1. Project Timeline & 15-Day Milestone Deadlines	1
2. Biological Background & Context	2
3. Primary Paper Reading Guide	2
Key Reading Focus & Guided Questions	2
4. Academic Integrity & AI Usage Policy	2
Mandatory AI Usage Guidelines	2
5. Environment Management with uv (Fast & Reproducible)	3
1. Installing uv	3
2. Initializing the Project Environment	3
3. Locking Dependencies for Team Collaboration	3
6. Research Questions to Address in Your Report	4
7. Dataset Overview	4
8. Step-by-Step Python Analysis Walkthrough	5
Step 8.1: Loading Libraries & 10x Data into Scanpy	5
Step 8.2: Quality Control (QC) & Cell Filtering	6
Step 8.3: Normalization, HVG Selection & Scaling	7
Step 8.4: Dimensionality Reduction (PCA & UMAP) & Clustering	7
Step 8.5: Cell Type Annotation & Marker Discovery	7
Step 8.6: Deep-Dive into EGFP & Müller Glia Dynamics	8
9. Report Structure & Expected Deliverables	8

1. Project Timeline & 15-Day Milestone Deadlines

This project has a **15-day execution timeline**, starting **Today, August 25, 2026** and concluding on **September 9, 2026 at 11:59 PM BST**:

Milestone	Phase & Key Tasks	Active Period	Hard Deadline	Key Deliverables
Milestone 1	Draft Report & Pipeline Tag	Aug 25 – Aug 31, 2026	Aug 31, 2026 (11:59 PM BST)	Tag v1.0-peer-review + Draft PDF
Milestone 2	Peer Review on GitHub Issues	Sep 01 – Sep 04, 2026	Sep 04, 2026 (11:59 PM BST)	Post Review on Assigned Group Issue
Milestone 3	Rebuttal & Final Submission	Sep 05 – Sep 09, 2026	Sep 09, 2026 (11:59 PM BST)	Tag v2.0-final + Final Report PDF

2. Biological Background & Context

Unlike mammals, the teleost zebrafish (*Danio rerio*) possesses a remarkable capacity to regenerate damaged retinal tissue and restore visual function. Following photoreceptor ablation, **Müller glia (MG)**—the primary radial glial cells of the retina—dedifferentiate, re-enter the cell cycle, and divide asymmetrically to produce multipotent progenitor cells that subsequently differentiate into replacement photoreceptors (rods and cones) and other retinal neurons.

In this project, photoreceptors were selectively ablated using the chemical toxin **MNU (N-methyl-N-nitrosourea)**. To trace cellular reprogramming in real time, transgenic zebrafish carrying the *careg:EGFP* reporter construct were used. The *careg* regulatory element becomes actively transcribed specifically in responding Müller glia upon injury, making **EGFP** a direct genetic indicator of regenerative activation.

3. Primary Paper Reading Guide

The primary reference paper for this project is provided in your folder: [paper.pdf](#) (GEO: GSE202212).

Before and during your computational analysis, read through [paper.pdf](#) with focus on the following key sections:

Key Reading Focus & Guided Questions

Experimental Design (Abstract & Introduction)

- What is the biological rationale for using MNU to induce photoreceptor ablation?
- How does the *careg* regulatory element function, and why is EGFP used as a surrogate marker for regeneration?

Cell Ranger & Single-Cell Sequencing Methods

- How were retinal cells dissociated and prepared for the 10x Chromium 3' library?
- How was the custom **EGFP** sequence integrated into the zebrafish reference genome for read alignment?

Figures to Compare with Your Analysis

- **Figure 1 & 2:** Retinal cell atlas, UMAP clusters, and canonical marker genes. Compare your cluster annotations against the authors' cell-type definitions.
- **Figure 3 & 4:** *careg:EGFP* kinetics across uninjured (Ctrl), 3dp, 7dp, and 10dp timepoints. Does your Scanpy violin/feature plot match the reported peak expression at 3–7 days post-injury?
- **Supplementary Tables:** Look for the list of validated retina cell marker genes provided in the supplementary material to support your cell-type annotations.

4. Academic Integrity & AI Usage Policy

Generative AI tools (e.g., ChatGPT, Claude, Gemini, GitHub Copilot) may be used as **learning and programming aids** during this project, subject to the following mandatory transparency rules:

Mandatory AI Usage Guidelines

1. Transparency & Prompt Sharing

- If you use AI to write code, debug errors, assist in pipeline design, or interpret biological results, you **MUST include an “AI Usage Disclosure / Appendix”** in your final report.
- In this appendix, you must document:
 - The AI tool and model used (e.g., *Claude 3.5 Sonnet*, *ChatGPT-4o*).

- The exact **prompts** you submitted.
- The purpose of each prompt (e.g., “*Debugging Scanpy Leiden clustering parameter error*” or “*Finding canonical zebrafish cone opsin genes*”).

2. Critical Verification & Authorship

- AI outputs can hallucinate gene names, biological functions, or code syntax. You are personally responsible for verifying the accuracy of every line of code, parameter choice, and biological claim.
- Simply copy-pasting AI-generated text or interpretations into your report without critical validation constitutes academic misconduct.

3. Code Understanding

- You must be able to explain every step of your Python script during project evaluation/oral defense.

5. Environment Management with uv (Fast & Reproducible)

We use **uv** (an extremely fast, modern Python package and project manager written in Rust) instead of Conda. **uv** installs packages in seconds and ensures 100% reproducible environments.

1. Installing uv

- **Linux / macOS / WSL:**

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- **Windows (PowerShell):**

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- **Via pip:**

```
pip install uv
```

2. Initializing the Project Environment

In your project repository directory:

```
# 1. Create a virtual environment with Python 3.11
uv venv --python 3.11

# 2. Activate the virtual environment:
# On Linux/macOS/WSL:
source .venv/bin/activate
# On Windows PowerShell:
.venv\Scripts\Activate.ps1

# 3. Install core single-cell packages instantly with uv:
uv pip install scanpy anndata pandas numpy scipy matplotlib seaborn gseapy jupyter ipykernel
```

3. Locking Dependencies for Team Collaboration

Generate a pinned lockfile or requirements file so your team members and peer reviewers can reproduce your exact environment in seconds:

```
uv pip freeze > requirements.txt
```

When team members clone your repository, they can recreate your exact environment instantly:

```
git clone https://github.com/YourUsername/sc-project1-zebrafish-retina.git
cd sc-project1-zebrafish-retina
uv venv
source .venv/bin/activate # (.venv\Scripts\Activate.ps1 on Windows)
uv pip install -r requirements.txt
```

Alternatively, you can run scripts directly without manual activation using `uv run`:

```
uv run workflow.py
```

6. Research Questions to Address in Your Report

As a student researcher, your goal is to analyze the provided single-cell RNA-seq data and answer the following core scientific questions in your report:

1. **Cell Type Identification:** What are the major cell types present in the adult zebrafish retina (e.g., Rods, Cones, Müller Glia, Bipolar cells, Amacrine cells, Retinal Ganglion cells)?
2. **Kinetics of the Regenerative Response:** How does the cellular composition shift across the post-injury timeline (Control → 3 days → 7 days → 10 days post-injury)?
3. **EGFP Reporter Dynamics:** Which specific cell clusters express `EGFP`, at which timepoint does `EGFP` reach maximum intensity, and does expression subside by day 10?
4. **Müller Glia Reprogramming:** Can you identify distinct sub-states of Müller glia (quiescent MG vs. activated MG vs. proliferating progenitor cells)?
5. **Photoreceptor Vulnerability & Recovery:** How do gene expression profiles in Rods and Cones change in response to injury?

7. Dataset Overview

Inside your project folder `project1_zebrafish_retina/`, the data is organized as follows:

```
data/
├─ ctrl1/filtered_feature_bc_matrix/ # Uninjured Control - Replicate 1
├─ ctrl2/filtered_feature_bc_matrix/ # Uninjured Control - Replicate 2
├─ 3dp1/filtered_feature_bc_matrix/  # 3 Days Post-Injury - Replicate 1
├─ 3dp2/filtered_feature_bc_matrix/  # 3 Days Post-Injury - Replicate 2
├─ 7dp1/filtered_feature_bc_matrix/  # 7 Days Post-Injury - Replicate 1
├─ 7dp2/filtered_feature_bc_matrix/  # 7 Days Post-Injury - Replicate 2
├─ 10dp1/filtered_feature_bc_matrix/ # 10 Days Post-Injury - Replicate 1
└─ 10dp2/filtered_feature_bc_matrix/ # 10 Days Post-Injury - Replicate 2
```

Each sample directory contains standard 10x output files: - `barcodes.tsv.gz`: Cell barcode identifiers - `features.tsv.gz`: Gene IDs and gene symbols (including `EGFP`) - `matrix.mtx.gz`: Sparse count matrix (genes × cells) - `web_summary.html`: Cell Ranger sequencing and alignment summary

8. Step-by-Step Python Analysis Walkthrough

Step 8.1: Loading Libraries & 10x Data into Scanpy

```
import scanpy as sc
import anndata as ad
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# Configure plotting parameters
sc.settings.verbosity = 3
sc.settings.set_figure_params(dpi=120, facecolor="white", frameon=False)
```

Load all 8 samples into a single unified AnnData object:

```
data_dir = Path("data")

# Locate all sample folders containing 10x matrices
sample_dirs = sorted([
    p for p in data_dir.iterdir()
    if p.is_dir() and (p / "filtered_feature_bc_matrix").exists()
])

print(f"Discovered {len(sample_dirs)} samples: {[p.name for p in sample_dirs]}")

adatas = {}
for sample_path in sample_dirs:
    sample_name = sample_path.name
    matrix_dir = sample_path / "filtered_feature_bc_matrix"

    # Read 10x matrix
    adata_sample = sc.read_10x_mtx(
        path=matrix_dir,
        var_names="gene_symbols",
        cache=False
    )

    # Assign metadata annotations
    adata_sample.obs["sample"] = sample_name

    # Parse condition (ctrl, 3dp, 7dp, 10dp) and replicate (1, 2)
    if sample_name.startswith("ctrl"):
        adata_sample.obs["condition"] = "ctrl"
        adata_sample.obs["replicate"] = sample_name.replace("ctrl", "")
    elif "dp" in sample_name:
        cond, rep = sample_name.split("dp")
        adata_sample.obs["condition"] = f"{cond}dp"
        adata_sample.obs["replicate"] = rep
    else:
```

```

adata_sample.obs["condition"] = sample_name
adata_sample.obs["replicate"] = "1"

adatas[sample_name] = adata_sample

# Merge all samples into one AnnData object
adata = ad.concat(adatas, label="sample_batch", index_unique="-", join="outer")
adata.var_names_make_unique()

print(f"Total dataset dimensions: {adata.n_obs} cells x {adata.n_vars} genes")

```

Step 8.2: Quality Control (QC) & Cell Filtering

In zebrafish (*Danio rerio*), gene symbols follow lowercase conventions: - **Mitochondrial genes**: start with `mt-` (e.g., `mt-co1`, `mt-nd2`) - **Ribosomal genes**: start with `rps` or `rpl` - **Hemoglobin genes**: start with `hba`, `hbb`, `hb-`

```

# 1. Identify QC gene families
adata.var["mt"] = adata.var_names.str.startswith("mt-")
adata.var["ribo"] = adata.var_names.str.startswith(("rps", "rpl"))
adata.var["hb"] = adata.var_names.str.startswith(("hba", "hbb", "hb-"))

# 2. Calculate QC metrics
sc.pp.calculate_qc_metrics(
    adata,
    qc_vars=["mt", "ribo", "hb"],
    percent_top=None,
    loglp=False,
    inplace=True
)

# 3. Plot QC violin distributions
sc.pl.violin(
    adata,
    ["n_genes_by_counts", "total_counts", "pct_counts_mt", "pct_counts_ribo"],
    groupby="condition",
    jitter=0.4,
    multi_panel=True
)

# 4. Filter low-quality cells and lowly expressed genes
min_genes_threshold = 200
min_cells_threshold = 3
max_mt_pct = 15.0

sc.pp.filter_cells(adata, min_genes=min_genes_threshold)
sc.pp.filter_genes(adata, min_cells=min_cells_threshold)
adata = adata[adata.obs["pct_counts_mt"] < max_mt_pct, :].copy()

print(f"Cells remaining after QC filtering: {adata.n_obs}")

```

Step 8.3: Normalization, HVG Selection & Scaling

```
# Store raw counts in a separate layer
adata.layers["counts"] = adata.X.copy()

# Library size normalization (10,000 counts per cell) and log1p transformation
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
adata.raw = adata # Save normalized/log1p data in .raw

# Identify Highly Variable Genes (HVGs)
sc.pp.highly_variable_genes(
    adata,
    n_top_genes=2000,
    batch_key="sample"
)
sc.pl.highly_variable_genes(adata)

# Regress out unwanted technical noise (total counts, mitochondrial percentage)
sc.pp.regress_out(adata, ["total_counts", "pct_counts_mt"])
sc.pp.scale(adata, max_value=10)
```

Step 8.4: Dimensionality Reduction (PCA & UMAP) & Clustering

```
# Compute PCA
sc.tl.pca(adata, svd_solver="arpack", use_highly_variable=True)
sc.pl.pca_variance_ratio(adata, log=True, n_pcs=30)

# Build k-nearest neighbor graph (adjust n_pcs based on the elbow plot)
sc.pp.neighbors(adata, n_neighbors=15, n_pcs=20)

# Compute 2D UMAP embedding
sc.tl.umap(adata)

# Unsupervised Leiden clustering
sc.tl.leiden(adata, resolution=0.6, key_added="leiden_res_0.6")

# Visualize UMAP
sc.pl.umap(adata, color=["leiden_res_0.6", "condition", "sample"], ncols=2)
```

Step 8.5: Cell Type Annotation & Marker Discovery

Use canonical retina gene markers from the literature to annotate each cluster:

Cell Type	Canonical Marker Genes	Description
Müller Glia (Resting)	rlbp1a, glul1a, slc1a3b	Quiescent radial glia providing retinal support
Activated Müller Glia / Progenitors	EGFP, pcna, mki67, ascl1a, her4.1	Dedifferentiating, proliferating regenerative cells

Cell Type	Canonical Marker Genes	Description
Rod Photoreceptors	<code>rho</code> , <code>gnat1</code> , <code>nr2e3</code>	Dim-light photoreceptors sensitive to MNU damage
Cone Photoreceptors	<code>opn1sw1</code> (UV), <code>opn1sw2</code> (Blue), <code>opn1mw1</code> (Green), <code>opn1lw1</code> (Red)	Daylight/color photoreceptors
Bipolar Cells	<code>vsx1</code> , <code>vsx2</code> , <code>islet1</code>	Interneurons transmitting signals from photoreceptors
Amacrine Cells	<code>gad1b</code> , <code>gad2</code> , <code>tfap2a</code>	Inhibitory interneurons
Retinal Ganglion Cells (RGCs)	<code>rbpms2b</code> , <code>is12b</code> , <code>pou4f1</code>	Projection neurons forming optic nerve
Microglia / Immune Cells	<code>mpeg1.1</code> , <code>coro1a</code> , <code>cxcr4b</code>	Resident retinal macrophages

```
# 1. Calculate cluster marker genes
sc.tl.rank_genes_groups(adata, groupby="leiden_res_0.6", method="wilcoxon")
sc.pl.rank_genes_groups(adata, n_genes=8, sharey=False)

# 2. Inspect canonical marker gene expression on UMAP
canonical_markers = [
    "EGFP", "rho", "gnat1", "opn1sw1", "opn1mw1",
    "rlbpla", "glula", "pcna", "mki67", "vsx1", "gad1b", "rbpms2b"
]
available_markers = [g for g in canonical_markers if g in adata.var_names]

sc.pl.umap(adata, color=available_markers, ncols=3)
sc.pl.dotplot(adata, var_names=available_markers, groupby="leiden_res_0.6")
```

Step 8.6: Deep-Dive into EGFP & Müller Glia Dynamics

```
# 1. Check EGFP expression across injury timepoints
if "EGFP" in adata.var_names:
    sc.pl.violin(adata, keys="EGFP", groupby="condition", rotation=45)
    sc.pl.umap(adata, color="EGFP", cmap="Reds")

# 2. Cell proportion analysis over time
cell_counts = pd.crosstab(adata.obs["condition"], adata.obs["leiden_res_0.6"],
normalize="index") * 100
cell_counts.plot(kind="bar", stacked=True, figsize=(10, 6), colormap="tab20")
plt.ylabel("Cell Proportion (%)")
plt.title("Cluster Proportions Across Regeneration Timepoints")
plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.show()
```

9. Report Structure & Expected Deliverables

Your student report should be formatted as a scientific article with the following sections:

1. **Title & Abstract** (~200 words): Summary of background, methods, key findings, and biological conclusions.
2. **Introduction**: Retinal structure, regenerative mechanisms in zebrafish vs. mammals, and purpose of the *careg:EGFP* reporter.
3. **Methods**: Description of the Scanpy pipeline (QC criteria, normalization parameters, HVG count, PCA dimensions, clustering resolution).
4. **Results**:
 - **Figure 1**: QC violin plots and cell filtering summary.
 - **Figure 2**: UMAP embeddings colored by cluster, condition, and sample replicate.
 - **Figure 3**: Dotplot / heatmap of marker genes and cluster cell-type assignments.
 - **Figure 4**: *careg:EGFP* dynamics and proliferation markers (*pcna*, *mki67*) across timepoints.
 - **Figure 5**: Quantification of cell-type proportion shifts during regeneration.
5. **Discussion**: Biological interpretation of the timeline. When is peak MG activation observed? How do Rods vs. Cones recover? Comparison with published findings in [paper.pdf](#).
6. **References**: Cited literature.
7. **Appendix: AI Usage Disclosure**: Full list of AI tools used, exact prompts submitted, purpose, and validation notes.